LAPORAN PRAKTIKUM

Struktur Data

MODUL 5

SINGLY LINKED LIST (BAGIAN KEDUA)



Disusun oleh:

Elsa Dwi Rizqiyanti

109082500090

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 5 : Struktur-Data/Modul5 at main · elsadwiyt/Struktur-Data

1. singlylist.h

Source Code

```
#ifndef SINGLYLIST_H
#define SINGLYLIST_H

#include <iostream>
#define Nil NULL

using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertFirst(List &L, address P);

address findElm(List L, infotype x);
int totalInfo(List L);

#endif
```

Deskripsi program

Program Singlylist.h ini berfungsi sebagai header buat mendeklarasikan struktur Singly Linked List bertipe integer di C++. Di sini didefinisikan bentuk node, pointer, dan penampung list-nya. Isinya juga mencakup deklarasi fungsi-fungsi dasar kayak buat bikin list, alokasi memori, nambah data di awal, cetak data, sampai fungsi tambahan buat nyari data dan ngehitung total isi list.

2. **Singlylist.cpp**

Source Code

```
#include "Singlylist.h"

void createList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        P->info = x;
        P->next = Nil;
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void insertFirst(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
    } else {
        P->next = L.First;
        L.First = P;
    }
}

void printInfo(List L) {
    address P = L.First;
    while (P != Nil) {
        cout << P->info << " ";
        P = P->next;
    }
    cout << endl;
}

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil) {
        if (P->info == x) {
            return P;
        }
        P = P->next;
    }
```

```
    return Nil;
}

int totalInfo(List L) {
    int total = 0;
    address P = L.First;
    while (P != Nil) {
        total += P->info;
        P = P->next;
    }
    return total;
}
```

Deskripsi program

Program Singlylist.cpp ini berisi implementasi logika dari seluruh fungsi dan prosedur yang sebelumnya dideklarasikan di Singlylist.h. Ini juga diatur cara inisialisasi list kosong, alokasi memori node, penghapusan memori, penambahan data di awal list, hingga pencetakan seluruh isi list. Selain itu, ada juga implementasi logika untuk mencari elemen tertentu di dalam list serta menjumlahkan total nilai dari semua node.

3. **Main.cpp**

Source Code

```
#include <iostream>
#include "Singlylist.h"

using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5 = Nil;
    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

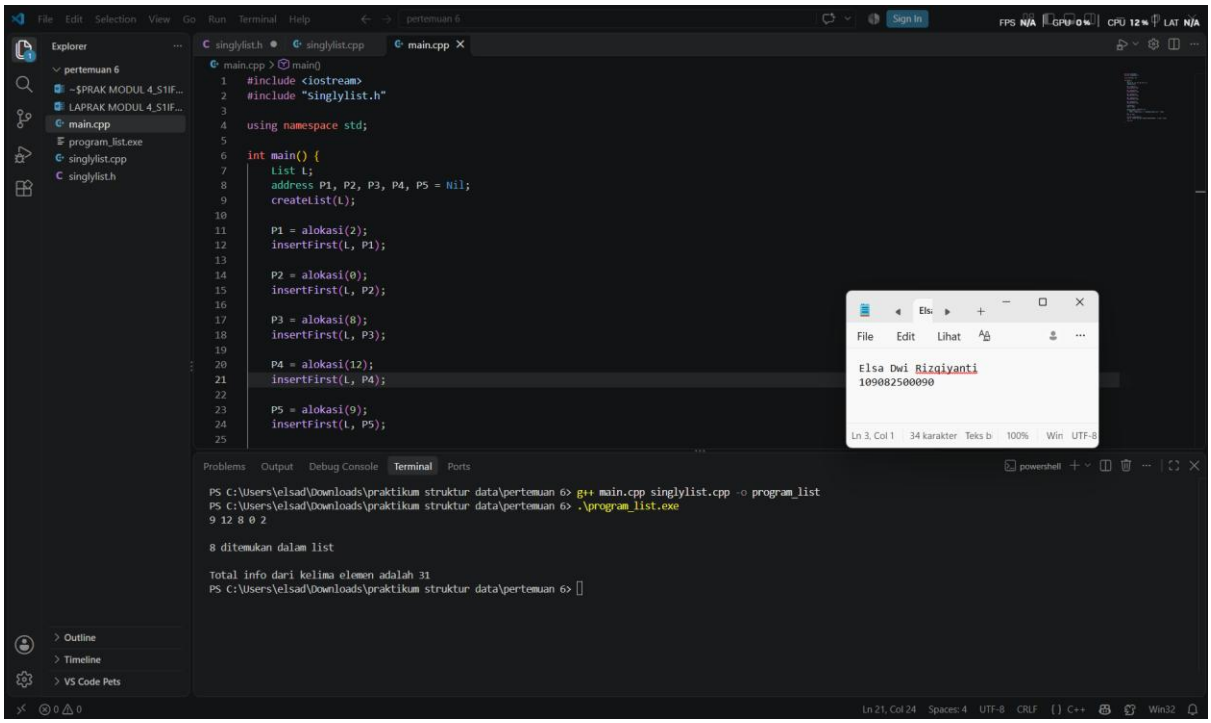
    printInfo(L);
    cout << endl;

    address found = findElm(L, 8);
    if (found != Nil) {
        cout << found->info << " ditemukan dalam list" <<
endl;
    }
    cout << endl;

    int sum = totalInfo(L);
    cout << "Total info dari kelima elemen adalah " << sum <<
endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Program main.cpp ini bertindak sebagai program utama untuk menjalankan dan menguji fungsi-fungsi linked list yang sudah dibuat. Kode ini menginisialisasi list kosong, mengalokasikan lima elemen data (2, 0, 8, 12, 9), lalu memasukkannya satu per satu ke awal list menggunakan insertFirst. Setelah itu, program menampilkan urutan isi list ke layar, menjalankan pencarian elemen bernilai 8 menggunakan findElm, serta menghitung dan menampilkan total penjumlahan seluruh angka di dalam list menggunakan totalInfo.

4. **SingleLinkedList.h (guided)**

Source Code

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk me
void dealokasi(ElmList* &P);
void printInfo(List L);

// Searching
ElmList* findElm(List L, int nilai);
// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

// Delete
void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

// Operasi tambahan
void update(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);

#endif
```

Deskripsi program

Program SLL.h ini berfungsi sebagai file header yang memuat deklarasi struktur data Singly Linked List bertipe integer beserta fungsi-fungsi pengolahannya yang cukup lengkap. Di dalamnya terdapat definisi struktur node dan penampung list, serta purwarupa untuk fungsi dasar, pencarian, berbagai teknik penyisipan data, penghapusan node, hingga operasi tambahan seperti mengubah nilai, menghitung total elemen, dan menghapus seluruh isi list.

5. SingleLinkedList.cpp (guided)

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}

// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;

    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}
```

```
// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    P->next = L.First;
    L.First = P;
}

// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    if (L.First == nullptr) {
        L.First = P;
    } else {
        ElmList* Q = L.First;

        while (Q->next != nullptr) {
            Q = Q->next;
        }

        Q->next = P;
    }
}

// Menambahkan node setelah nilai tertentu
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
    ElmList* Q = findElm(L, nilaiSebelum);

    if (Q == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else {
```

```
            ElmList* P = alokasi(nilaiBaru);

            P->next = Q->next;
            Q->next = P;
        }
    }

    // Menghapus node pertama
    void delFirst(List &L) {
        if (L.First == nullptr) {
            cout << "List kosong." << endl;
        } else {
            ElmList* P = L.First;

            L.First = L.First->next;
            dealokasi(P);
        }
    }

    // Menghapus node terakhir
    void delLast(List &L) {
        if (L.First == nullptr) {
            cout << "List kosong." << endl;
        } else if (L.First->next == nullptr) {
            delFirst(L);
        } else {
            ElmList* P = L.First;

            while (P->next->next != nullptr) {
                P = P->next;
            }

            ElmList* Q = P->next;
            P->next = nullptr;
            dealokasi(Q);
        }
    }

    // Menghapus node setelah nilai tertentu
    void delAfter(List &L, int nilaiSebelum) {
        ElmList* P = findElm(L, nilaiSebelum);

        if (P == nullptr) {
            cout << "Nilai sebelumnya tidak ditemukan." << endl;
        } else if (P->next == nullptr) {
            cout << "Tidak ada node setelah nilai tersebut." <<
    endl;
        } else {
```

```
            ElmList* Q = P->next;

            P->next = Q->next;
            dealokasi(Q);
        }
    }

    // Menghapus node berdasarkan nilai
    void delP(List &L, int nilai) {
        if (L.First == nullptr) {
            cout << "List kosong." << endl;
        } else if (L.First->info == nilai) {
            delFirst(L);
        } else {
            ElmList* P = L.First;

            while (P->next != nullptr &&
                   P->next->info != nilai) {
                P = P->next;
            }

            if (P->next == nullptr) {
                cout << "Nilai tidak ditemukan." << endl;
            } else {
                ElmList* Q = P->next;

                P->next = Q->next;
                dealokasi(Q);
            }
        }
    }

    // Mengubah nilai data
    void update(List &L, int nilaiLama, int nilaiBaru) {
        ElmList* P = findElm(L, nilaiLama);

        if (P == nullptr) {
            cout << "Nilai tidak ditemukan." << endl;
        } else {
            P->info = nilaiBaru;
        }
    }

    // Menghitung jumlah node
    int nbList(List L) {
        int jumlah = 0;
        ElmList* P = L.First;
```

```
        while (P != nullptr) {
            jumlah++;
            P = P->next;
        }

        return jumlah;
    }

    // Menghapus semua node
    void delAll(List &L) {
        while (L.First != nullptr) {
            delFirst(L);
        }
    }
```

Deskripsi program

Program SingleLinkedList.cpp ini berisi implementasi logika lengkap dari seluruh fungsi dan prosedur yang sebelumnya dideklarasikan di file header. Kode ini mengatur fungsi dasar (pembuatan list, alokasi/dealokasi memori, pencetakan data), pencarian elemen, serta berbagai manipulasi data seperti penambahan (insertFirst, insertLast, insertAfter), penghapusan (delFirst, delLast, delAfter, delP), pengubahan nilai (update), penghitungan jumlah node (nbList), hingga pembersihan seluruh isi list (delAll).

6. Main.cpp (guided)

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* hasil;
    ElmList* P;
    int total = 0;

    CreateList(L);

    // 1. Insert First
    insertFirst(L, 2);
    insertFirst(L, 0);
    insertFirst(L, 8);
    insertFirst(L, 12);
    insertFirst(L, 9);

    cout << "List awal: ";
    printInfo(L);
```

```
        // 2. Searching
        hasil = findElm(L, 8);

        if (hasil != nullptr) {
            cout << "Angka 8 ditemukan." << endl;
        } else {
            cout << "Angka 8 tidak ditemukan." << endl;
        }

        // 3. Menghitung total nilai
        P = L.First;

        while (P != nullptr) {
            total = total + P->info;
            P = P->next;
        }

        cout << "Total nilai: " << total << endl;

        // 4. Insert Last
        insertLast(L, 20);
        cout << "Setelah insertLast(20): ";
        printInfo(L);

        // 5. Insert After
        insertAfter(L, 12, 99);
        cout << "Setelah insertAfter(12, 99): ";
        printInfo(L);

        // 6. Update
        update(L, 99, 100);
        cout << "Setelah update(99, 100): ";
        printInfo(L);

        // 7. Delete After
        delAfter(L, 12);
        cout << "Setelah delAfter(12): ";
        printInfo(L);

        // 8. Delete berdasarkan nilai
        delP(L, 8);
        cout << "Setelah delP(8): ";
        printInfo(L);

        // 9. Delete First
        delFirst(L);
        cout << "Setelah delFirst: ";
        printInfo(L);
```

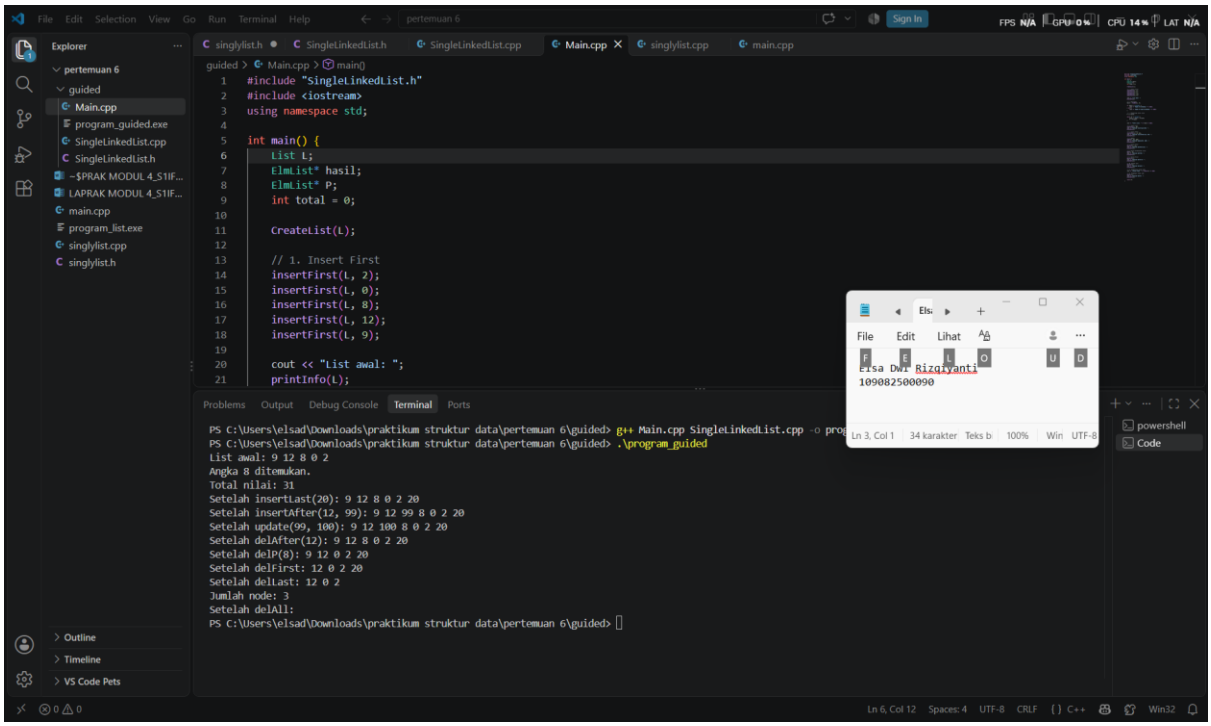
```
    // 10. Delete Last
    delLast(L);
    cout << "Setelah delLast: ";
    printInfo(L);

    // 11. Menghitung jumlah node
    cout << "Jumlah node: " << nbList(L) << endl;

    // 12. Menghapus semua node
    delAll(L);
    cout << "Setelah delAll: ";
    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program Main.cpp ini berfungsi sebagai program utama untuk menguji seluruh fitur manipulasi data pada Singly Linked List. Program diawali dengan menginisialisasi list, menyisipkan data awal dengan insertFirst, melakukan pencarian data (findElm), serta menghitung total nilainya. Setelah itu, program mendemonstrasikan berbagai operasi lanjutan secara berurutan, seperti penambahan data (insertLast, insertAfter), pembaruan nilai (update), berbagai metode penghapusan node (delAfter, delP, delFirst, delLast), hingga menghitung jumlah sisa node (nbList) dan mengosongkan seluruh isi list menggunakan delAll.